# Python Exercise: Build a Calculator

A progressive, hands-on exercise. Each level builds on the previous one, so work through them in order. Type the code yourself rather than copying — the mistakes are where the learning happens.

**Prerequisites:** Python 3.8+ installed. Check with `python3 --version`.

**Estimated time:** 3–5 hours across all levels.

---

## Learning objectives

By the end of this exercise you will be able to:

- Define and call functions with parameters and return values
- Read and validate user input
- Use conditionals and loops to control program flow
- Handle errors with `try` / `except` instead of letting the program crash
- Organise code into modules and write automated tests
- Read a traceback and fix the bug it points to

---

## Level 1 — Four operations

### Goal

A program that asks for two numbers and an operator, then prints the result.

### Tasks

1. Create a file named `calculator.py`.
2. Write four functions: `add(a, b)`, `subtract(a, b)`, `multiply(a, b)`, `divide(a, b)`. Each takes two numbers and **returns** the result — do not use `print()` inside them.
3. Ask the user for the first number, the operator, and the second number.
4. Call the right function and print the result.

### Starter code

```
def add(a, b):
    return a + b


def subtract(a, b):
    # TODO: return a minus b
    pass


def multiply(a, b):
    # TODO
    pass


def divide(a, b):
    # TODO
    pass


first = float(input("First number: "))
operator = input("Operator (+ - * /): ")
second = float(input("Second number: "))

# TODO: choose the correct function based on `operator` and print the result
```

### Expected output

```
First number: 12
Operator (+ - * /): *
Second number: 4
12.0 * 4.0 = 48.0
```

### Checkpoint questions

- Why do the functions `return` instead of `print`? What could you do with a returned value that you cannot do with a printed one?
- What does `float()` do to the string that `input()` gives you? What happens without it?

> **Hint (Level 1)** `input()` always returns a string. `"12" + "4"` is `"124"`, not `16`. Converting with `float()` is what makes arithmetic work.

---

## Level 2 — Do not let it crash

Right now your program dies on bad input. Fix that.

### Tasks

1. **Division by zero.** `divide(5, 0)` raises `ZeroDivisionError`. Catch it and print a clear message instead.
2. **Non-numeric input.** Typing `hello` at a number prompt raises `ValueError`. Catch it and ask again.
3. **Invalid operator.** If the user types `%` or `x`, tell them what the valid options are.
4. Write a helper function `read_number(prompt)` that keeps asking until it gets a valid number, then returns it.

### Starter code

```
def read_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("That is not a number. Try again.")


def divide(a, b):
    # TODO: handle b == 0 without crashing
    return a / b
```

### Test cases

Your program must survive all of these:

| Input | Expected behaviour |
|---|---|
| `10`, `/`, `0` | Message about dividing by zero, no crash |
| `abc` at a number prompt | Re-prompts, no crash |
| `5`, `%`, `3` | Message listing valid operators |
| `-7.5`, `+`, `2.5` | `-5.0` |
| `1e3`, `*`, `2` | `2000.0` |

### Checkpoint questions

- What is the difference between catching `ValueError` and checking `if not text.isdigit()` first? Which handles `-3.5` correctly?

- Why is `except Exception:` a worse choice than `except ValueError:`?

> **Hint (Level 2)** Two ways to handle division by zero: check `if b == 0:` before dividing, or wrap the division in `try` / `except ZeroDivisionError`. Both are valid. Think about which reads more clearly.

---

## Level 3 — Keep going

A calculator you have to restart after every sum is annoying.

### Tasks

1. Wrap the whole thing in a loop so it runs repeatedly.
2. Add a menu. Let the user type `q` or `quit` to exit.
3. Move the operator dispatch into a **dictionary** instead of a long `if` / `elif` chain.
4. Extract the single-calculation logic into a function `calculate(a, operator, b)` that returns the result.

### Dictionary dispatch

This is the key idea in this level. Instead of:

```
if operator == "+":
    result = add(a, b)
elif operator == "-":
    result = subtract(a, b)
# ... and so on
```

store the functions in a dictionary and look one up:

```
OPERATIONS = {
    "+": add,
    "-": subtract,
    "*": multiply,
    "/": divide,
}

def calculate(a, operator, b):
    if operator not in OPERATIONS:
        raise ValueError(f"Unknown operator: {operator}")
    return OPERATIONS[operator](a, b)
```

Note that `OPERATIONS["+"]` gives you the *function itself*, not a call to it. Adding the `()` afterwards is what runs it.

### Expected output

```
=== Calculator ===
Enter a calculation, or 'q' to quit.

First number: 8
Operator: +
Second number: 5
8.0 + 5.0 = 13.0

First number: q
Goodbye.
```

### Checkpoint questions

- What do you have to change to add a `%` (modulo) operator now? Compare that to what you would have changed in Level 1.
- What is the advantage of `calculate()` returning a value rather than printing it? (Think ahead to Level 5.)

---

## Level 4 — Memory and history

### Tasks

1. Keep a list of every calculation performed this session.
2. Add a `history` command that prints them, numbered.
3. Add an `ans` keyword: typing `ans` as a number uses the previous result.
4. Add a `clear` command that empties the history.

### Expected output

```
First number: 10
Operator: +
Second number: 5
10.0 + 5.0 = 15.0

First number: ans
Operator: *
Second number: 2
15.0 * 2.0 = 30.0

First number: history
  1. 10.0 + 5.0 = 15.0
  2. 15.0 * 2.0 = 30.0
```

### Design question

Where should the history live? Three options:

- A global list
- A parameter passed into every function
- A `Calculator` class holding it as an attribute

Try the global list first because it is quickest, then refactor to a class. Write down what actually improved — if nothing did, say so. Not every refactor is worth it, and noticing that is a real skill.

> **Hint (Level 4)** Store each history entry as a tuple `(a, operator, b, result)` rather than a pre-formatted string. That way you can reformat it later, or reuse the numbers — a formatted string throws that information away.

---

## Level 5 — Extensions

Pick whichever interest you. These are open-ended.

### 5a. More operations

Add power (`**`), integer division (`//`), and modulo (`%`). Then add square root, using the `math` module. What should `sqrt` do about the second operand — and what should it do with a negative input?

### 5b. Expression parsing

Instead of three separate prompts, accept a whole expression:

```
> 12 * 4
48.0
```

Start by splitting on whitespace. Then consider: what breaks with `12*4` (no spaces)? What about `2 + 3 * 4` — should it give `20` or `14`? Correct operator precedence is genuinely hard, and discovering *why* is the point of this task.

### 5c. Command-line interface

Make it work as `python3 calculator.py 12 '*' 4`, using `sys.argv`. Then rewrite it with `argparse` and compare.

### 5d. Persistent history

Save history to a JSON file and reload it on startup. Consider what should happen if the file is missing or corrupt.

---

## Level 6 — Tests

Manual testing does not scale. Automate it.

### Tasks

1. Create `test_calculator.py`.
2. Write tests for each operation, including edge cases.
3. Run with `python3 -m unittest test_calculator.py -v`.

### Starter code

```
import unittest
from calculator import add, subtract, multiply, divide, calculate


class TestOperations(unittest.TestCase):

    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-2, -3), -5)

    def test_divide_by_zero_raises(self):
        with self.assertRaises(ZeroDivisionError):
            divide(5, 0)

    def test_float_precision(self):
        # 0.1 + 0.2 != 0.3 in floating point — this is why assertAlmostEqual exists
        self.assertAlmostEqual(add(0.1, 0.2), 0.3)

    # TODO: subtract, multiply, divide
    # TODO: calculate() with an unknown operator


if __name__ == "__main__":
    unittest.main()
```

### Checkpoint questions

- Run `python3 -c "print(0.1 + 0.2)"`. Why is the answer not exactly `0.3`? What does that mean for a calculator handling money?
- What is a test that passes but tells you nothing useful?

---

## Common errors and what they mean

| Error | Usual cause |
|---|---|
| `TypeError: unsupported operand type(s) for +: 'str' and 'str'` | Forgot `float()` on the input |
| `ValueError: could not convert string to float` | User typed something non-numeric |
| `ZeroDivisionError: division by zero` | Second operand was 0 in a division |
| `KeyError: 'x'` | Looked up an operator not in `OPERATIONS` — use `.get()` or check membership first |
| `TypeError: 'NoneType' object is not ...` | A function fell off the end without returning — check for a missing `return` |
| `IndentationError` | Mixed tabs and spaces, or inconsistent indent depth |

---

## Self-assessment

You have finished when you can answer yes to all of these:

- ☐ The program never crashes, whatever the user types
- ☐ Adding a new operator means changing exactly one line
- ☐ Every function has a single clear job and returns a value
- ☐ Tests pass, and you added at least one that failed first and then passed after a fix
- ☐ You can explain every line to someone else without hedging

---

## Reference solution — Levels 1 to 3

Do not read this until you have made a serious attempt. Struggling with a bug for twenty minutes teaches more than reading a working answer.

```
"""A simple command-line calculator."""


def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b


OPERATIONS = {
    "+": add,
    "-": subtract,
    "*": multiply,
    "/": divide,
}


def calculate(a, operator, b):
    """Apply `operator` to `a` and `b`. Raises ValueError on an unknown operator."""
```

```
    if operator not in OPERATIONS:
        valid = " ".join(OPERATIONS)
        raise ValueError(f"Unknown operator '{operator}'. Valid: {valid}")
    return OPERATIONS[operator](a, b)


def read_number(prompt):
    """Prompt until the user enters a valid number. Returns None if they quit."""
    while True:
        text = input(prompt).strip()
        if text.lower() in ("q", "quit", "exit"):
            return None
        try:
            return float(text)
        except ValueError:
            print(f"  '{text}' is not a number. Try again, or 'q' to quit.")


def main():
    print("=== Calculator ===")
    print("Enter a calculation, or 'q' to quit.\n")

    while True:
        first = read_number("First number: ")
        if first is None:
            break

        operator = input("Operator: ").strip()

        second = read_number("Second number: ")
        if second is None:
            break

        try:
            result = calculate(first, operator, second)
        except ZeroDivisionError as exc:
            print(f"  Error: {exc}\n")
            continue
        except ValueError as exc:
            print(f"  Error: {exc}\n")
            continue

        print(f"{first} {operator} {second} = {result}\n")

    print("Goodbye.")


if __name__ == "__main__":
    main()
```

**Points worth noticing in this solution:**

- `if __name__ == "__main__":` means the file can be imported by the test module without `main()` running.
- `divide` raises rather than printing. The function's job is arithmetic; deciding what the user sees belongs to the caller.
- `continue` after an error returns to the top of the loop instead of printing a broken result.
- `.strip()` removes the trailing space or newline users often leave behind.
- The error message lists the valid operators, so the user does not have to guess.